

Branching

References

- BH chapter 3

Branching

- CPU executes program sequentially
- However, control can be transferred to other parts.
- Transfers can be conditional or unconditional
- Conditional Transfers:
 - Control is transferred to a new location if a certain condition is true
- Unconditional Transfers:
 - Control is transferred to a new location in all cases.
- The program will start running sequentially from where the control was transferred.

Branching in HLL

- Following is an example of conditional transfer in C++

1. If (x!=0)

```
{  
    Z=Y/X  
}
```

2. while (c<10)

```
{  
    sum= a[c--]  
}
```

3. goto in C++ can be used for unconditional jump

COMPARISON AND CONDITIONS

- The basic root instruction for all comparisons is CMP standing for compare.
- The operation of CMP is to subtract the source operand from the destination operand, updating the flags without changing either the source or the destination.
- CMP is one of the key instructions as it introduces the capability of conditional routing in the processor.

COMPARISON AND CONDITIONS

- subtraction can check many different conditions.
- For example if a larger number is subtracted from a smaller number then borrow is needed. The carry flag plays the role of borrow during the subtraction operation. And in this condition the carry flag will be set.
- If two equal numbers are subtracted the answer is zero and the zero flag will be set.
- Every significant relation between the destination and source is evident from the sign flag, carry flag, zero flag, and the overflow flag.
- CMP is meaningless without a conditional jump immediately following it.

COMPARISON AND CONDITIONS

- In unsigned numbers only the magnitude of the number is important, whereas in signed numbers both the magnitude and the sign are important. For example -2 is greater than -3 but 2 is smaller than 3.
- Inside the computer signed numbers are represented in **two's complement notation**.
- For signed numbers JG and JL will work properly and for unsigned JA and JB will work properly
- The unsigned comparisons see the numbers as 0 being the smallest and 65535 being the largest with the order that $0 < 1 < 2 \dots < 65535$.
- For signed $-32768 < -32767 < \dots < -1 < 0 < 1 < 2 < \dots < 32767$

DEST = SRC	ZF = 1	When the source is subtracted from the destination and both are equal the result is zero and therefore the zero flag is set. This works for both signed and unsigned numbers.			positive number signals that the destination is smaller.
UDEST < USRC	CF = 1	When an unsigned source is subtracted from an unsigned destination and the destination is smaller, borrow is needed which sets the carry flag.	SDEST ≤ SSRC	ZF = 1 OR SF ≠ OF	If the zero flag is set, it means that the source and destination are equal and if the sign and overflow flags differ it means that the destination is smaller as described above.
UDEST ≤ USRC	ZF = 1 OR CF = 1	If the zero flag is set, it means that the source and destination are equal and if the carry flag is set it means a borrow was needed in the subtraction and therefore the destination is smaller.	SDEST ≥ SSRC	SF = OF	When a signed source is subtracted from a signed destination and the answer is positive with no overflow than the destination is greater than the source. When an overflow is there signaling that sign has changed unexpectedly, we interpret a negative answer as the signal that the destination is greater.
UDEST ≥ USRC	CF = 0	When an unsigned source is subtracted from an unsigned destination no borrow will be needed either when the operands are equal or when the destination is greater than the source.			
UDEST > USRC	ZF = 0 AND CF = 0	The unsigned source and destination are not equal if the zero flag is not set and the destination is not smaller since no borrow was taken. Therefore the destination is greater than the source.			
SDEST < SSRC	SF ≠ OF	When a signed source is subtracted from a signed destination and the answer is negative with no overflow than the destination is smaller than the source. If however there is an overflow meaning that the sign has changed unexpectedly, the meanings are reversed and a	SDEST > SSRC	ZF = 0 AND SF = OF	If the zero flag is not set, it means that the signed operands are not equal and if the sign and overflow match in addition to this it means that the destination is greater than the source.

Unconditional Jump in Assembly

- JMP is used for unconditional jump to another part of code
- Format
 - JMP <destination>
 - Where destination can be any 16 bit address where your desired instruction is located.
 - Usually it's a code label

- Example:

```
; Unconditional Jump example
[org 0x0100]
    mov ax=0
    jmp start ; this will transfer the control to code label start
    add ax, 1
    add ax, 2
start:
    add ax, 3
    add ax, 4
    mov ax, 0x4c00
    int 0x21
```

Unconditional Jump in Assembly

- Especially used when data is defined within code

```
; Unconditional Jump example
[org 0x0100]

    jmp start ; this will transfer the control to code label start

; defining data
my_array: db 1,2,3,4,57,8

start: ; program will start running from here
    add ax, 3
    add ax, 4
mov ax, 0x4c00
int 0x21
```

Question:

- What is the following code doing?

```
; Unconditional Jump example
```

```
[org 0x0100]
```

```
11:
```

```
    add ax, 3
```

```
    jmp 11
```

```
mov ax, 0x4c00
```

```
int 0x21
```

Conditional Jumps

- Conditional jump are implemented a combination of comparisons and jumps.
- It's a two step process:
 - First, an operation such as CMP, AND, or SUB modifies the CPU status flags.
 - Second, a conditional jump instruction tests the flags and causes a branch to a new address.
- Example of code snippet

```
...  
cmp cx, 5  
je 11  
...  
  
11:  
...
```

Example

```
; if bx>5 add to ax  
[org 0x0100]
```

```
mov bx, 6  
mov ax, 6  
cmp bx, 5  
jl terminate; will not take the jump  
add ax, bx
```

terminate:

```
mov ax, 0x4c00  
int 0x21
```

```
; if bx>5 add to ax  
[org 0x0100]
```

```
mov bx, 3  
mov ax, 6  
cmp bx, 5  
jl terminate; will take the jump  
;next line will not be executed  
add ax, bx
```

terminate:

```
mov ax, 0x4c00  
int 0x21
```

Cmp instruction

- The CMP (compare) instruction performs an implied subtraction of a source operand from a destination operand.
- Neither operand is modified
- The CMP instruction changes the Overflow, Sign, Zero, Carry, Auxiliary Carry, and Parity flags according to the value the destination operand would have had if actual subtraction had taken place.

Conditional jump instruction

- In last examples you saw je and jl instructions.
- These are example of conditional jump instructions
- Je stands for jump if equal, jl stands for jump if less.
- There are many other conditional jumps.
- These instructions uses flags to check whether to take or not take the jump.
 - For example:
 - je will check ZF to see if the numbers were equal or not.
 - cmp would have set ZF to 1 is both operands were equal
- Format is
 - `<conditional jump instruction> <destination>`

Types of Conditional jumps

- Jumps based on specific flag values
- Jumps based on equality between operands or the value of CX
- Jumps based on comparisons of unsigned operands
- Jumps based on comparisons of signed operands

Types of Conditional jumps

Table 6-2 Jumps Based on Specific Flag Values.

Mnemonic	Description	Flags / Registers
JZ	Jump if zero	ZF = 1
JNZ	Jump if not zero	ZF = 0
JC	Jump if carry	CF = 1
JNC	Jump if not carry	CF = 0
JO	Jump if overflow	OF = 1
JNO	Jump if not overflow	OF = 0
JS	Jump if signed	SF = 1
JNS	Jump if not signed	SF = 0
JP	Jump if parity (even)	PF = 1
JNP	Jump if not parity (odd)	PF = 0

Example

```
; if bx is not zero then add it to ax  
[org 0x0100]
```

```
mov ax, 5;  
mov bx, 0;
```

```
cmp bx, 0  
jz terminate; jump will be taken  
add ax, bx
```

terminate:

```
mov ax, 0x4C00  
int 21h
```

```
; if bx is not zero then add it to ax  
[org 0x0100]
```

```
mov ax, 5;  
mov bx, 1;
```

```
cmp bx, 0  
jz terminate; jump will not be taken  
add ax, bx
```

terminate:

```
mov ax, 0x4C00  
int 21h
```

Example

```
; a program to add ten numbers
[org 0x0100]

mov bx, num1 ; point bx to first number
mov cx, 10 ; load count of numbers in cx
mov ax, 0 ; initialize sum to zero

l1:
add ax, [bx] ; add number to ax
add bx, 2 ; advance bx to next number
sub cx, 1 ; numbers to be added reduce
jnz l1 ; if numbers remain add next

mov [total], ax ; write back sum in memory

mov ax, 0x4c00 ; terminate program
int 0x21

num1: dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50
total: dw 0
```

Example

```
; add two 16 bit numbers and store output 3 bytes memory
; use the MSB will be carry (i.e 00h or 01h)

[org 0x0100]
jmp start

num1: dw 0xFFFF
num2: dw 0xFFFF ; change the data to see different results
output: dw 0; for output
        db 0; for carry

start:
    mov ax, [num1]
    mov bx, [num2]
    add ax, bx
    jnc write; if no carry leave the MSB byte as zero and jump to writing 2 bytes
    mov byte [output+2],1

write:
    mov word [output], ax

mov ax, 0x4C00
int 21h
```

Types of Conditional jumps

Table 6-3 Jumps Based on Equality.

Mnemonic	Description
JE	Jump if equal (<i>leftOp = rightOp</i>)
JNE	Jump if not equal (<i>leftOp $\neq$ rightOp</i>)
JCXZ	Jump if CX = 0

Example

```
; add first 10 +ve integers in ax  
[org 0x0100]
```

```
mov ax, 0  
mov bx, 0
```

```
l1:
```

```
  add ax, bx ; add number to ax  
  add bx, 1 ; advance bx to next integer  
  cmp bx, 11 ; compared bx with 11  
  jne l1 ; if bx is 11 do not take the jump
```

```
mov ax, 0x4c00  
int 0x21
```

Types of Conditional jumps

Table 6-4 Jumps Based on Unsigned Comparisons.

Mnemonic	Description
JA	Jump if above (if $leftOp > rightOp$)
JNBE	Jump if not below or equal (same as JA)
JAЕ	Jump if above or equal (if $leftOp \geq rightOp$)
JNB	Jump if not below (same as JAЕ)
JB	Jump if below (if $leftOp < rightOp$)
JNAЕ	Jump if not above or equal (same as JB)
JBE	Jump if below or equal (if $leftOp \leq rightOp$)
JNA	Jump if not above (same as JBE)

Types of Conditional jumps

Table 6-5 Jumps Based on Signed Comparisons.

Mnemonic	Description
JG	Jump if greater (if $leftOp > rightOp$)
JNLE	Jump if not less than or equal (same as JG)
JGE	Jump if greater than or equal (if $leftOp \geq rightOp$)
JNL	Jump if not less (same as JGE)
JL	Jump if less (if $leftOp < rightOp$)
JNGE	Jump if not greater than or equal (same as JL)
JLE	Jump if less than or equal (if $leftOp \leq rightOp$)
JNG	Jump if not greater (same as JLE)

Difference Between Signed and Unsigned number

- The processor does not consider the difference between signed or unsigned number
- It only maintains flags for either case
- It depends on programmer how they interpret the flag and which jump instructions they use.

Example

```
2                                     [org 0x0100]
3
4 00000000 B8FEFF                    mov ax, -2
5 00000003 050100                    add ax, 1
6
7 00000006 B8004C                    mov ax, 0x4c00
8 00000009 CD21                      int 0x21
```

```
2                                     [org 0x0100]
3
4 00000000 B8FEFF                    mov ax, 65534
5 00000003 050100                    add ax, 1
6
7 00000006 B8004C                    mov ax, 0x4c00
8 00000009 CD21                      int 0x21
```

- Two different assembly codes
- Same machine code
- Same ax after code ends
 - Ax=FFFF
- Processor will on SF in both cases
 - SF=1
- It depends on the programmer to interpret AX as 65535 or as -1

Example

- Difference between jumps for signed and unsigned comparison.
- After cmp, CF=1, SF=1, OF=1

```
[org 0x0100]
```

```
mov al, 125
```

```
cmp al, -126
```

```
ja IsAbove ; JUMP WILL NOT BE TAKEN AS -128<127
```

```
jg IsGreater; JUMP WILL BE TAKEN AS IT CONSIDERS NUMBERS as UNSIGNED
```

```
IsAbove:
```

```
    mov ax, 1
```

```
    jmp terminate
```

```
IsGreater:
```

```
    mov ax, 2
```

```
    jmp terminate
```

```
terminate:
```

```
mov ax, 0x4c00
```

```
int 0x21
```

```
[org 0x0100]
```

```
    mov al, 7Dh;125 o  
    cmp al, 82h;can be -126 or 130  
    ja  IsAbove ;  
    jg  IsGreater;
```

```
IsAbove:
```

```
    mov ax, 1  
    jmp terminate
```

```
IsGreater:
```

```
    mov ax, 2  
    jmp terminate
```

```
terminate:
```

```
    mov ax, 0x4c00  
    int 0x21
```

Example

Dec	Hex	Binary
125	7D	0111 1101
-126	82	1000 0010
$125 - (-126) = 251$	$7D - 82 = FB$	1111 1011

- CF is on because 7D-82 needs borrow
- SF is on because MSB of answer is 1
- OF flag is on because, 251 is out of range of -128 to 127
 - You can also determine OF by looking at the difference in MSB of 1st operand and result (i.e. 7D and FB, they are different)

Near Jump

- **Near Jump** :When the relative address stored with the instruction is in 16 bits as in the last example the jump is called a near jump.
- Using a near jump we can jump anywhere within a segment. If we add a large number it will wrap around to the lower part.
- A negative number actually is a large number and works this way using the wraparound behavior

Short Jump

- If the offset is stored in a single byte as in 75F2 with the opcode 75 and operand F2, the jump is called a short jump. F2 is added to IP as a signed byte. If the byte is negative the complement is negated from IP otherwise the byte is added. Unconditional jumps can be short, near, and far.
- Conditional jumps can only be short. A short jump can go +127 bytes ahead in code and -128 bytes backwards and no more. This is the limitation of a byte in signed representation

Far Jump

- Far jump is not position relative but is absolute.
- Both segment and offset must be given to a far jump.
- Sometimes we may need to go from one code segment to another, and near and short jumps cannot take us there.
- Far jump must be used and a two byte segment and a two byte offset are given to it.
- It loads CS with the segment part and IP with the offset part.
- Execution therefore resumes from that location in physical memory. The three instructions that have a far form are JMP, CALL, and RET, are related to program control.
- Far capability makes intra segment control possible.

Example 3.1

```
001      ; a program to add ten numbers without a separate counter
002      [org 0x0100]
003          mov     bx, 0                ; initialize array index to zero
004          mov     ax, 0                ; initialize sum to zero
005
006      ll:      add     ax, [num1+bx]    ; add number to ax
007              add     bx, 2            ; advance bx to next index
008              cmp     bx, 20           ; are we beyond the last index
009              jne     ll               ; if not add next number
010
011              mov     [total], ax      ; write back sum in memory
012
013              mov     ax, 0x4c00       ; terminate program
014              int     0x21
015
016      num1:     dw     10, 20, 30, 40, 50, 10, 20, 30, 40, 50
017      total:    dw     0
```

Example 3.2

```
001      ; a program to add ten numbers without a separate counter
002      [org 0x0100]
003              jmp start                      ; unconditionally jump over data
004
005      num1:      dw    10, 20, 30, 40, 50, 10, 20, 30, 40, 50
006      total:     dw    0

007
008      start:      mov    bx, 0                ; initialize array index to zero
009                  mov    ax, 0                ; initialize sum to zero
010
011      ll:         add    ax, [num1+bx]         ; add number to ax
012                  add    bx, 2                ; advance bx to next index
013                  cmp    bx, 20               ; are we beyond the last index
014                  jne    ll                   ; if not add next number
015
016                  mov    [total], ax          ; write back sum in memory
017
018                  mov    ax, 0x4c00           ; terminate program
019                  int     0x21
```

003 **JMP** jumps over the data declarations to the start label and execution resumes from there.

SORTING EXAMPLE

Example 3.3

```
001 ; sorting a list of ten numbers using bubble sort
002 [org 0x0100]
003         jmp  start
004
005 data:    dw  60, 55, 45, 50, 40, 35, 25, 30, 10, 0
006 swap:    db  0
007
008 start:    mov  bx, 0                ; initialize array index to zero
009          mov  byte [swap], 0      ; rest swap flag to no swaps
010
011 loop1:    mov  ax, [data+bx]       ; load number in ax
012          cmp  ax, [data+bx+2]     ; compare with next number
013          jbe  noswap              ; no swap if already in order
014
015          mov  dx, [data+bx+2]     ; load second element in dx
016          mov  [data+bx+2], ax     ; store first number in second
017          mov  [data+bx], dx       ; store second number in first
018          mov  byte [swap], 1      ; flag that a swap has been done
019
020 noswap:    add  bx, 2                ; advance bx to next index
021          cmp  bx, 18               ; are we at last index
022          jne  loop1               ; if not compare next two
023
024          cmp  byte [swap], 1      ; check if a swap has been done
025          je   start               ; if yes make another pass
026
027          mov  ax, 0x4c00           ; terminate program
028          int  0x21
```

Homework

```
data:      dw 60, 55, 45, 50, -40, -35, 25, 30, 10, 0
```

FLAGS associate with jumps

- You can see all these association in table given in BH page 42